

# EPFL CIVIL-127, Lab 8 solution

## binary\_search\_tree\_no\_recursion.py

```
# Binary search tree implementation without using recursion
#
# Is this version simpler/easier to read than the recursive version?

class BinarySearchTree:
    def __init__(self) -> None:
        self.root: Node | None = None

    def serialize(self) -> str:
        """
        Useful for debugging and unittesting!
        """
        if self.root is None:
            return "<>"
        return self.root.serialize()

    def is_valid_bst(self) -> bool:
        """
        Helper method to check that the tree is a valid bst
        """
        if self.root is not None:
            return self.root.is_valid_bst(None, None)
        return True

    def add(self, value: int) -> None:
        c = self.root
        if c is None:
            # Case where the tree was empty
            self.root = Node(value)
            return

        # Notice how the type of c from here onwards is no longer Node | None
        # and it's just Node. This is called type narrowing.

        while True:
            if value < c.getValue():
```

```

        next_c = c.getLeft()
        if next_c is None:
            # Value goes in a new left child
            c.setLeft(Node(value))
            return
        c = next_c
    elif value > c.getValue():
        next_c = c.getRight()
        if next_c is None:
            # Value goes in a new right child
            c.setRight(Node(value))
            return
        c = next_c
    else:
        # value == c.getValue() is implied
        # We are done, the value was already in the tree
        return

def contains(self, value: int) -> bool:
    c = self.root
    while c is not None:
        if value < c.getValue():
            c = c.getLeft()
        elif value > c.getValue():
            c = c.getRight()
        else:
            # value == c.getValue() is implied
            return True
    return False

def remove(self, value: int) -> None:
    # Remove is tricky since there are a few cases, including moving nodes around.
    previous = self.root
    if previous is None:
        # the tree is empty, we have nothing to do
        return
    if value == previous.getValue():
        # Remove the root node
        left = previous.getLeft()
        right = previous.getRight()
        if left is None:
            # There's no left node, promote the right subtree to root
            self.root = right

```

```

        return

    if left.getRight() is None:
        # Promote the left node and give it a right subtree
        self.root = left
        left.setRight(right)
        return

    # Remove largest value in the left subtree and promote it to root
    promote = self.remove_largest(left)
    promote.setLeft(left)
    promote.setRight(right)
    self.root = promote
    return

while True:
    # Search for value by keeping track of the previous node
    # so that we can update the nodes correctly
    if value < previous.getValue():
        c = previous.getLeft()
        if c is None:
            # the node was not found
            return
        if value == c.getValue():
            # remove c and update previous.left
            left = c.getLeft()
            right = c.getRight()
            if left is None:
                # There's no further left node, promote the right subtree
                previous.setLeft(right)
                return
            if left.getRight() is None:
                # Promote the left node and give it a right subtree
                previous.setLeft(left)
                left.setRight(right)
                return
            # Remove largest value in the left subtree and promote it
            promote = self.remove_largest(left)
            promote.setLeft(left)
            promote.setRight(right)
            previous.setLeft(promote)
            return
        previous = c
        continue
    elif value > previous.getValue():

```

```

        c = previous.getRight()
        if c is None:
            # the node was not found
            return
        if value == c.getValue():
            # remove c and update previous.right
            left = c.getLeft()
            right = c.getRight()

            if left is None:
                # There's no further left node, promote the right subtree
                previous.setRight(right)
                return
            if left.getRight() is None:
                # Promote the left node and give it a right subtree
                previous.setRight(left)
                left.setRight(right)
                return
            # Remove largest value in the left subtree and promote it
            promote = self.remove_largest(left)
            promote.setLeft(left)
            promote.setRight(right)
            previous.setRight(promote)
            return
        previous = c
        continue

    # if we get here, that means our code is seriously broken. We failed to
    # both find and not find the value.
    assert False

def remove_largest(self, node: Node) -> Node:
    """Removes the right most leaf and return it"""

    c = node
    while True:
        right = c.getRight()
        assert right is not None
        if right.getRight() is None:
            c.setRight(None)
            return right
        c = right

```

```

def smallest(self) -> int:
    # Compare the implementation of smallest with largest, they are two different
    # approaches to have code which type checks correctly.
    c = self.root
    if c is None:
        raise IndexError("tree is empty")
    while True:
        next_c = c.getLeft()
        if next_c is None:
            return c.getValue()
        c = next_c

def largest(self) -> int:
    c = self.root
    if c is None:
        raise IndexError("tree is empty")

    # if we don't use a `while True` loop, we have to deal with a potentially None
    value for c
    while c is not None and c.getRight() is not None:
        c = c.getRight()

    # And the type system isn't smart enough to know that c can't be None here so we
    # need to deal with a potential None again.
    assert c is not None
    return c.getValue()

class Node:
    def __init__(self, value: int):
        self._value = value
        self.left = None
        self.right = None

    def serialize(self) -> str:
        r = "<"
        if self.left is not None:
            r += self.left.serialize() + ":"
        r += str(self._value)
        if self.right is not None:
            r += ":" + self.right.serialize()
        r += ">"
        return r

```

```

def is_valid_bst(self, min: int | None, max: int | None) -> bool:
    if min is not None and self._value <= min:
        return False
    if max is not None and self._value >= max:
        return False

    ok = True
    if self.left is not None:
        ok = ok and self.left.is_valid_bst(min, self._value)
    if self.right is not None:
        ok = ok and self.right.is_valid_bst(self._value, max)
    return ok

def getValue(self) -> int:
    return self._value

def setLeft(self, child: Node | None) -> None:
    self.left = child

def getLeft(self) -> Node | None:
    return self.left

def setRight(self, child: Node | None) -> None:
    self.right = child

def getRight(self) -> Node | None:
    return self.right

```

## binary\_search\_tree\_with\_recursion.py

```

# Binary search tree implementation with recursion.
#
# Is this version simpler/easier to read than the non-recursive version? It seems
# shorter. Most of the logic moves to the Node class

class BinarySearchTree:
    def __init__(self) -> None:
        self.root: Node | None = None

```

```

def serialize(self) -> str:
    """
    Useful for debugging and unittesting!
    """
    if self.root is None:
        return "<>"
    return self.root.serialize()

def is_valid_bst(self) -> bool:
    """
    Helper method to check that the tree is a valid bst
    """
    if self.root is not None:
        return self.root.is_valid_bst(None, None)
    return True

def add(self, value: int) -> None:
    if self.root is None:
        # Case where the tree was empty
        self.root = Node(value)
    else:
        # Notice how the type of self.root from here onwards is no longer Node | None
        # and it's just Node. This is called type narrowing.
        self.root.add(value)

def contains(self, value: int) -> bool:
    if self.root is None:
        # Empty tree doesn't contain any values
        return False
    return self.root.contains(value)

def remove(self, value: int) -> None:
    if self.root is not None:
        self.root = self.root.remove(value)

def smallest(self) -> int:
    if self.root is None:
        raise IndexError("tree is empty")
    return self.root.smallest()

def largest(self) -> int:
    if self.root is None:
        raise IndexError("tree is empty")

```

```

        return self.root.largest()

class Node:
    def __init__(self, value: int):
        self._value = value
        self.left: Node | None = None
        self.right = None

    def serialize(self) -> str:
        r = "<"
        if self.left is not None:
            r += self.left.serialize() + ":"
        r += str(self._value)
        if self.right is not None:
            r += ":" + self.right.serialize()
        r += ">"
        return r

    def is_valid_bst(self, min: int | None, max: int | None) -> bool:
        if min is not None and self._value <= min:
            return False
        if max is not None and self._value >= max:
            return False

        ok = True
        if self.left is not None:
            ok = ok and self.left.is_valid_bst(min, self._value)
        if self.right is not None:
            ok = ok and self.right.is_valid_bst(self._value, max)
        return ok

    def getValue(self) -> int:
        return self._value

    def setLeft(self, child: Node | None) -> None:
        self.left = child

    def getLeft(self) -> Node | None:
        return self.left

    def setRight(self, child: Node | None) -> None:
        self.right = child

```



```

def getRight(self) -> Node | None:
    return self.right

def add(self, value: int) -> None:
    if value == self._value:
        # we already have this value, no need to do anything
        return
    elif value < self._value:
        if self.left is None:
            self.left = Node(value)
        else:
            self.left.add(value)
    elif value > self._value:
        if self.right is None:
            self.right = Node(value)
        else:
            self.right.add(value)

def contains(self, value: int) -> bool:
    if value < self._value:
        if self.left is None:
            return False
        else:
            return self.left.contains(value)
    elif value > self._value:
        if self.right is None:
            return False
        else:
            return self.right.contains(value)
    else:
        # value == self._value is implied
        return True

def remove(self, value: int) -> Node | None:
    """
    Finds the node with value and removes it. Returns the node which gets
    promoted in its place.
    """
    if value < self._value:
        if self.left is not None:
            self.left = self.left.remove(value)
        return self

```

```

elif value > self._value:
    if self.right is not None:
        self.right = self.right.remove(value)
    return self
else:
    # value == self._value is implied
    if self.left is None:
        # There's no left subtree, so the right subtree is promoted
        return self.right
    promote = self.left.remove_largest(self)
    promote.setLeft(self.left)
    promote.setRight(self.right)
    return promote

def remove_largest(self, parent: Node, first: bool = True) -> Node:
    """
    Find the largest value (node without a right child) and removes it from its
    parent.

    The first time remove_largest is called, we remove parent.left but as the code
    recurses,
    we remove parent.right (hence the first bool flag)
    """
    if self.right is not None:
        return self.right.remove_largest(self, False)
    if first:
        parent.left = None
    else:
        parent.right = None
    return self

def smallest(self) -> int:
    if self.left is None:
        return self._value
    return self.left.smallest()

def largest(self) -> int:
    if self.right is None:
        return self._value
    return self.right.largest()

```

# test\_binary\_search\_tree.py

```
import unittest

# from lab8.binary_search_tree_no_recursion import BinarySearchTree
from lab8.binary_search_tree_with_recursion import BinarySearchTree

class TestBinarySearchTree(unittest.TestCase):
    def tree(self) -> BinarySearchTree:
        bst = BinarySearchTree()
        self.assertTrue(bst.is_valid_bst())
        bst.add(8)
        self.assertTrue(bst.is_valid_bst())
        bst.add(10)
        self.assertTrue(bst.is_valid_bst())
        bst.add(14)
        self.assertTrue(bst.is_valid_bst())
        bst.add(13)
        self.assertTrue(bst.is_valid_bst())
        bst.add(3)
        self.assertTrue(bst.is_valid_bst())
        bst.add(6)
        self.assertTrue(bst.is_valid_bst())
        bst.add(7)
        self.assertTrue(bst.is_valid_bst())
        bst.add(4)
        self.assertTrue(bst.is_valid_bst())
        bst.add(1)
        self.assertTrue(bst.is_valid_bst())
        return bst

    def testAdd(self):
        bst = self.tree()
        self.assertEqual(bst.serialize(), "<<<1>:3:<<4>:6:<7>>>:8:<10:<<13>:14>>>")

    def testContains(self):
        bst = self.tree()
        values = (1, 3, 4, 6, 7, 8, 10, 13, 14)
        for i in range(0, 15):
            if i in values:
                self.assertTrue(bst.contains(i))
```

```

        else:
            self.assertFalse(bst.contains(i))

def testRemove(self):
    # Remove has a lot of different cases, so we need to write extensive tests

    bst = self.tree()
    bst.remove(1)
    self.assertEqual(bst.serialize(), "<<<3:<<4>:6:<7>>>:8:<10:<<13>:14>>>")
    self.assertFalse(bst.contains(1))
    self.assertTrue(bst.is_valid_bst())

    bst = self.tree()
    bst.remove(10)
    self.assertEqual(bst.serialize(), "<<<1>:3:<<4>:6:<7>>>:8:<<13>:14>>>")
    self.assertFalse(bst.contains(10))
    self.assertTrue(bst.is_valid_bst())

    bst = self.tree()
    bst.remove(14)
    self.assertEqual(bst.serialize(), "<<<1>:3:<<4>:6:<7>>>:8:<10:<13>>>")
    self.assertFalse(bst.contains(14))
    self.assertTrue(bst.is_valid_bst())

    bst = self.tree()
    bst.remove(8)
    self.assertEqual(bst.serialize(), "<<<1>:3:<<4>:6>>:7:<10:<<13>:14>>>")
    self.assertFalse(bst.contains(8))
    self.assertTrue(bst.is_valid_bst())

    bst = self.tree()
    bst.remove(5)
    self.assertEqual(bst.serialize(), "<<<1>:3:<<4>:6:<7>>>:8:<10:<<13>:14>>>")
    bst.remove(0)
    self.assertEqual(bst.serialize(), "<<<1>:3:<<4>:6:<7>>>:8:<10:<<13>:14>>>")
    self.assertTrue(bst.is_valid_bst())

    bst = BinarySearchTree()
    bst.add(8)
    bst.add(10)
    bst.add(14)
    bst.add(13)
    bst.remove(8)

```

```

self.assertEqual(bst.serialize(), "<10:<<13>:14>>")
self.assertFalse(bst.contains(8))
self.assertTrue(bst.is_valid_bst())

bst = BinarySearchTree()
bst.add(8)
bst.add(10)
bst.add(14)
bst.add(13)
bst.add(3)
bst.remove(8)
self.assertEqual(bst.serialize(), "<3:<10:<<13>:14>>>")
self.assertFalse(bst.contains(8))
self.assertTrue(bst.is_valid_bst())

bst = BinarySearchTree()
bst.add(8)
bst.add(3)
bst.add(1)
bst.add(10)
bst.remove(3)
self.assertEqual(bst.serialize(), "<<1>:8:<10>>")
self.assertFalse(bst.contains(3))
self.assertTrue(bst.is_valid_bst())

bst = BinarySearchTree()
bst.add(8)
bst.add(12)
bst.add(9)
bst.add(10)
bst.add(14)
bst.remove(12)
self.assertEqual(bst.serialize(), "<8:<<9>:10:<14>>>")
self.assertFalse(bst.contains(12))
self.assertTrue(bst.is_valid_bst())

bst = BinarySearchTree()
bst.remove(1)
self.assertEqual(bst.serialize(), "<>")
self.assertTrue(bst.is_valid_bst())

def testSmallest(self):
    bst = self.tree()

```

```
        self.assertTrue(bst.smallest(), 1)

        bst.remove(1)
        self.assertTrue(bst.smallest(), 3)
        self.assertTrue(bst.is_valid_bst())

    def testLargest(self):
        bst = self.tree()
        self.assertTrue(bst.largest(), 14)

        bst.remove(14)
        self.assertTrue(bst.smallest(), 13)
        self.assertTrue(bst.is_valid_bst())

if __name__ == "__main__":
    unittest.main()
```